Relatório 11 - Prática: Criando agentes com ADK e Streamlit (IV)

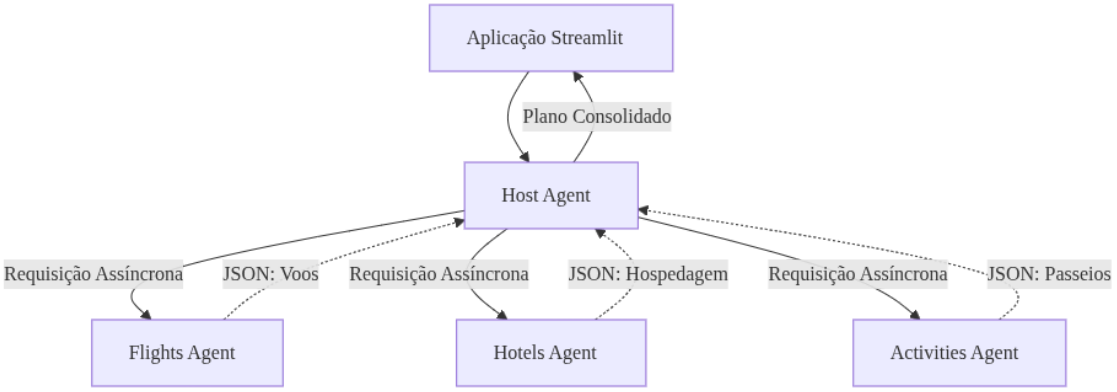
Lucas Augusto Nunes de Barros

Descrição da atividade

1. Desenvolvimento do Multi Agente de Viagens com o Google ADK

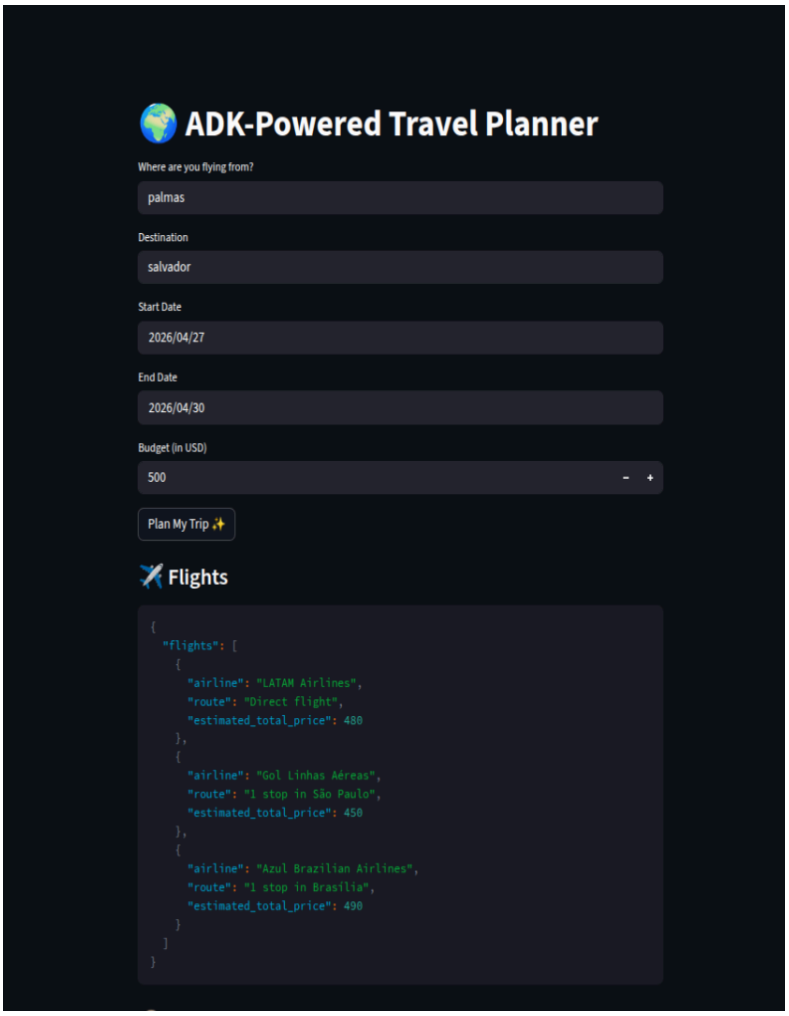
Nesta etapa prática, foi proposto o desenvolvimento de um ecossistema completo para planejamento de viagens operando sob uma arquitetura multi-agente distribuída. O sistema interage com o usuário final através de uma interface web construída em Streamlit e utiliza um agente orquestrador central para gerenciar e delegar tarefas para três sub agentes especializados.

Estrutura do Sistema Multi Agentes Proposto



A arquitetura proposta foi implementada utilizando comunicação via Agent-to-Agent (A2A) através de requisições HTTP. Cada um dos agentes foi instanciado como um micro serviço, essa abordagem simula um ambiente de produção mais próximo da realidade, além de garantir o isolamento e a escalabilidade de cada nó do sistema. Todos os agentes foram configurados para utilizar o modelo GPT4 da OpenAI usando a biblioteca LiteLLM.

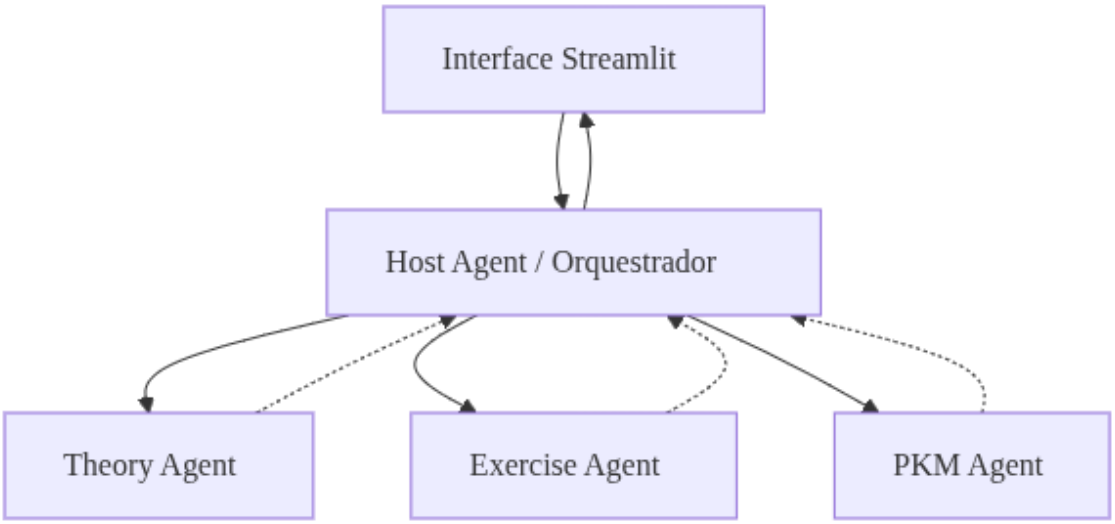
Sistema Proposto no Tutorial



2. Desenvolvimento do Sistema Multi Agente Planejador de Aulas

Semelhante à arquitetura desenvolvida anteriormente, foi implementado um gerador de aulas que executa agentes em paralelo. O orquestrador, que tem nome auto explicativo, orquestra o envio das tarefas e informações para cada um dos agentes. O primeiro agente (*theory agent*) busca a teoria e a apresenta de forma simplificada para facilitar o entendimento, um outro agente (*exercise agent*) é responsável pela elaboração de exercícios para fixação e consolidação da matéria estudada. Já o terceiro agente (*pkm agent*) reúne essas informações e as converte em um formato específico para facilitar a documentação via Obsidian.

Estrutura do Sistema Proposto



Esse tipo de abordagem promove o isolamento dos serviços/agentes resultando em um ecossistema modular, escalável e com carga otimizada. O formato dos dados foi garantido mediante o uso de saídas estruturadas, de grande importância para a integração de sistemas onde a saída de um agente é a entrada do outro. Para garantir que a documentação da aula seja perfeitamente consumida pelos plugins Dataview e TaskNotes, do Obsidian, o *pkm_agent* formata o conteúdo final estritamente em formato Markdown, usando os metadados YAML.

Implementação do Sistema Multi Agentes

Dificuldades

Durante a implementação e os testes de comunicação entre o orquestrador e os subagentes, surgiram alguns problemas no código que exigiram adaptações na estrutura proposta pelo tutorial.

Ao realizar a primeira solicitação via interface, o sistema retornou um erro de conexão (*httpx.ConnectError: All connection attempts failed)*. O erro ocorreu porque o *host_agent* tentou disparar o payload de dados, mas os os subagentes não estavam ativos. A solução foi criar um script shell para garantir que todos os processos do Uvicorn (flight_agent, stay_agent, activities_agent) fossem iniciados e estivessem escutando em suas respectivas portas antes que a interface do Streamlit e o orquestrador fossem liberados para uso.

Como os subagentes foram configurados com o armazenamento volátil na RAM (*InMemorySessionService*), o envio de múltiplas requisições usando o mesmo id da sessão (*SESSION_ID*) estava causando conflito e o sistema retornava um erro 500, informando que aquela sessão especificada no código não havia sido encontrada. Para solucionar esse problema, o código de cada sub agente foi reescrito para operar de forma *stateless*, ou seja, o agente não guarda nenhuma memória sobre as interações passadas. Com a geração de IDs únicos e dinâmicos para cada requisição e instanciando uma nova sessão a cada chamada o problema de não encontrar o *session id* foi resolvido.

Criação Dinâmica de SessionID

```
async def execute(request):
    SESSION_ID = f"session_stays_{uuid.uuid4().hex}"
```

Por último, durante a correção e criação dinâmica das sessões, uma das chamadas de sessão gerou um erro de sintaxe ao utilizar a palavra-chave *await* fora de uma função assíncrona. A correção exigiu o encapsulamento da lógica de inicialização de sessão para dentro do método *async def execute(request):*, garantindo que o *await* conseguisse pausar a execução até que o ADK registrasse o ID na memória antes de prosseguir.

Movendo *await* Para Função Assíncrona

```
async def execute(payload: dict):
    await session_service.create_session(app_name="theory_app",
```

Conclusões

A implementação dos sistemas propostos consolidou a base teórica sobre arquiteturas multi agentes distribuídas, aumentando a complexidade da estrutura ao introduzir uma comunicação Agent-to-Agent (A2A) baseada em microsserviços. A superação dos obstáculos técnicos durante a implementação deixou claro quais os requisitos para construir novos sistemas que transitam de um agente isolado para um ecossistema de agentes inteligentes que interagem entre si.

Referências

[1] CODELABS.DEVELOPERS.GOOGLE.COM. **Sistemas multiagentes com Agent2Agent | Google Codelabs**

Disponível em: <https://codelabs.developers.google.com/next26/adk-agent2agent?hl=pt-br#0>

Acesso em 27 de abril de 2026

[2] CLOUD.GOOGLE.COM. **O que é um sistema multiagente na IA? | Google Cloud**

Disponível em: <https://cloud.google.com/discover/what-is-a-multi-agent-system?hl=pt-BR>

Acesso em 27 de abril de 2026

[3] MEDIUM.COM. **A practical guide to building Multi-Agents AI Systems with A2A**

Disponível em <https://medium.com/google-cloud/a-practical-guide-to-building-multi-agents-ai-systems-with-a2a-2c0e3d77af24>

Acesso em 28 de abril de 2026

[4] DATACAMP.COM. **Google's Agent Development Kit (ADK): A Guide With Demo Project**

Disponível em: <https://www.datacamp.com/tutorial/agent-development-kit-adk>

Acesso em 26 de abril de 2026